

Design and Simulation of a Zero Trust Architecture for Secure Enterprise Networking

Autenia Murray

CSCI 7536: Network and Computer Security

Spring 2025

Abstract

This project designs and simulates a Zero Trust Architecture for secure enterprise networking. Traditional perimeter-based network security assumes that users, devices, and systems inside the network can be trusted after initial access is granted. That assumption creates security gaps because attackers, compromised devices, and insider threats may move laterally once they reach an internal network. Zero Trust addresses this weakness by requiring continuous verification, least privilege access, micro-segmentation, and ongoing monitoring for every access attempt.

The final implementation combines hands-on Azure Ubuntu lab work with a Python-based Zero Trust policy simulation. The Azure lab provided a real Linux environment for secure administration, packet capture, and traffic inspection. The virtual machine was protected through an Azure Network Security Group that restricted SSH access to a trusted public IP address. Inside the VM, a protected local HTTP resource was created on port 8080, test traffic was generated with curl and nmap, and network traffic was captured with tcpdump. The Python simulation modeled enterprise users, devices, network segments, protected resources, access policies, and monitoring alerts. The simulation evaluated 13 access requests and produced structured output files for access decisions, alerts, and test summaries. All 13 policy tests passed, with 3 requests allowed, 10 requests denied, and 13 alerts generated. The results show that the project successfully modeled core Zero Trust principles in a controlled and reproducible environment.

Introduction

Modern enterprise networks are no longer protected by a simple boundary between “inside” and “outside.” Users may connect from different locations, devices may be personally owned or unmanaged, cloud services may host sensitive data, and attackers may use stolen credentials to appear legitimate. In this environment, traditional perimeter-based security is not strong enough by itself. A firewall or virtual private network may control initial entry, but once an attacker gains access, internal systems may still trust that user or device too much.

Zero Trust Architecture responds to this problem by removing automatic trust from network location. The central idea is strict: no user, device, system, or network segment should be trusted by default. Every access request should be evaluated based on identity, device posture, resource sensitivity, least privilege, and monitoring context. A user should not be allowed to access a sensitive resource merely because the request came from inside the network.

This project addresses the security challenge of controlling access within an enterprise-style environment. The goal was to design and simulate a Zero Trust model that demonstrates micro-segmentation, authentication and authorization logic, device compliance checks, policy enforcement, and continuous monitoring. The project also includes real cloud lab evidence

through an Azure Ubuntu virtual machine, SSH restriction, Network Security Group configuration, security tool installation, packet capture, and live test traffic.

The final project was implemented as a hybrid simulation. The Azure Ubuntu VM provided a real Linux-based environment for network testing and monitoring. The Python simulation provided the Zero Trust decision engine and monitoring logic. Together, these components demonstrate both practical system setup and policy-based access control.

Problem Statement

The security problem addressed in this project is that traditional perimeter-based enterprise networks often grant too much trust after initial access. If a user account, endpoint, or internal system is compromised, an attacker may be able to move across internal resources because the network assumes that internal traffic is trustworthy. This creates risk for sensitive systems such as HR records, finance databases, and administrative consoles.

Justification

This problem matters because modern enterprise networks include cloud services, remote access, mobile devices, unmanaged endpoints, and users with different privilege levels. A single successful login or network connection should not automatically allow broad access to internal resources. A Zero Trust model reduces this risk by verifying each access request, limiting users to least privilege, checking device compliance, enforcing segmentation, and creating monitoring evidence for denied or suspicious activity.

Literature Review / Background

Zero Trust is a modern security model designed for environments where trust cannot be based only on physical location, network membership, or previous authentication. The National Institute of Standards and Technology describes Zero Trust as an approach in which access decisions are made dynamically and are based on identity, device state, policy, and other contextual signals. This model is especially important for cloud computing, remote work, hybrid networks, mobile devices, and distributed enterprise systems.

Traditional security models often depend on a strong outer perimeter. In that design, firewalls, VPNs, and boundary controls separate trusted internal networks from untrusted external networks. This approach can reduce some external threats, but it becomes weaker when attackers obtain credentials, compromise an endpoint, exploit an internal system, or abuse an insider account. Once inside, the attacker may be able to move laterally across internal resources if the network is too flat or if internal systems apply broad trust.

Existing Security Approaches

Existing enterprise security approaches commonly include firewalls, virtual private networks, access control lists, intrusion detection systems, multifactor authentication, endpoint protection, and network segmentation. Firewalls and VPNs help control entry into a network, while access control lists restrict traffic between systems. Identity and access management tools verify users, and endpoint security tools evaluate device risk. Monitoring tools such as intrusion detection systems and packet-capture utilities help identify suspicious traffic. These approaches are useful, but they are often weaker when used as separate controls instead of as part of a continuous verification model.

Zero Trust reduces this risk through several major principles. First, access should be granted using least privilege. Users should receive only the access needed for their role and task. Second, systems should verify identity and device posture before granting access. Third, networks should be segmented so that one compromised area does not expose the whole environment. Fourth, monitoring should be continuous, because trust is not a one-time decision. A user or device that appears safe at one moment may become risky later.

Micro-segmentation is one of the most important design concepts in this project. Instead of allowing broad movement across the network, resources are separated into smaller zones such as HR, Finance, IT Administration, Guest, and Public access areas. Access between these zones is controlled by policy. For example, an HR user may access HR records from a compliant device, but that same user should not be able to access the Finance database. This limits lateral movement and supports least privilege.

Device posture is another important Zero Trust concept. In a stronger access-control model, the system does not only ask, “Who is the user?” It also asks, “Is the device managed? Is it compliant? Is it encrypted? Is the risk score acceptable?” This is important because a valid user account may still be unsafe if the request comes from an unmanaged, noncompliant, or high-risk device.

Monitoring completes the Zero Trust model. Denied access attempts, repeated failures, suspicious device behavior, and cross-segment access attempts should be logged and reviewed. In this project, the monitoring engine records policy denials and generates alerts when repeated denied attempts occur. The Azure Ubuntu lab also demonstrates network-level monitoring by using tcpdump to capture real traffic generated against a protected test resource.

Position of This Project

This project fits within current Zero Trust practice by combining secure cloud setup, restricted remote administration, traffic monitoring, and policy-based access decisions. Instead of assuming that access is safe because it comes from inside a network, the simulation evaluates each request based on user role, account status, device compliance, device risk, network

segment, resource sensitivity, and monitoring behavior. This makes the project a small-scale academic model of the same Zero Trust principles used in modern enterprise security: verify explicitly, enforce least privilege, segment resources, and monitor continuously.

Methodology / Design

The project was designed around two connected components: a real Azure Ubuntu cloud lab and a Python-based Zero Trust simulation. The Azure environment provided a controlled Linux system for practical network testing. The Python simulation provided the policy engine, test scenarios, access decisions, logs, and alerts.

Cloud Lab Design

The Azure cloud lab included an Ubuntu 24.04 virtual machine, a virtual network and subnet, a network interface, a public IP address, a Network Security Group, and an SSH inbound rule restricted to a trusted public IP address.

The Ubuntu VM served as the technical environment for installing and verifying security tools. The Network Security Group supported secure network design by controlling inbound access. SSH was kept available on port 22 for administration, but the source was restricted from Any to a trusted IP address using /32 notation. This reduced unnecessary exposure while still allowing remote access to the VM.

▼

Deployment details

	Resource	Type	Status
✓	Zt	 Microsoft.Compute/virtualMach	OK
✓	zt61	 Microsoft.Network/networkInter	OK
✓	Zt-nsg	 Network security group	OK
✓	Zt-vnet	 Virtual network	OK
✓	Zt-ip	 Public IP address	OK

Figure 1. Azure cloud lab resources created for the Zero Trust project.

This screenshot shows the Azure resources created for the cloud lab, including the Ubuntu virtual machine, network interface, Network Security Group, virtual network, and public IP address. These resources provided the infrastructure used for secure access, network testing, packet capture, and Zero Trust simulation evidence.

Tools and Technologies

The project used a practical mix of cloud infrastructure, Linux tools, Python code, and monitoring utilities to simulate and evaluate the Zero Trust model.

Tool / Technology	Purpose in Project
Microsoft Azure	Hosted the Ubuntu virtual machine and cloud lab resources
Ubuntu 24.04 VM	Provided the Linux environment for implementation and testing
Azure Virtual Network	Represented the network environment for the lab
Azure Network Security Group	Enforced inbound SSH access restrictions
SSH	Provided secure remote administration of the VM
Python 3	Implemented the Zero Trust policy simulation
Python HTTP server	Hosted a protected local test resource on port 8080
nmap	Generated controlled service-detection traffic against the test resource
tcpdump	Captured and inspected packet traffic
CSV output files	Stored access logs, alerts, and test summaries

Security Strategies

The design integrated several Zero Trust security strategies. First, remote administration was protected by restricting SSH access through the Azure Network Security Group. Instead of leaving port 22 open to all internet addresses, the inbound rule was limited to a trusted public IP address using /32 notation. Second, the Python simulation enforced least privilege by allowing access only when the user’s role matched the protected resource. Third, the simulation modeled device posture by checking whether a device was managed, compliant, encrypted, and below the risk-score threshold. Fourth, micro-segmentation was represented through source-segment checks, which prevented users from accessing resources from unauthorized network segments. Fifth, monitoring was integrated through alert generation for denied requests and repeated failed access attempts.

Secure coding practices were also included in the simulation design. The code separates users, devices, resources, access requests, policy evaluation, and monitoring into clear structures. Each access decision records both the result and the reason for the result, which supports auditability.

The simulation does not hard-code one broad trust rule. Instead, it evaluates multiple conditions before allowing access. Encryption was represented through two layers: secure SSH administration for the Azure VM and simulated device-encryption checks for sensitive resources such as HR records, the Finance database, and the administrative console.

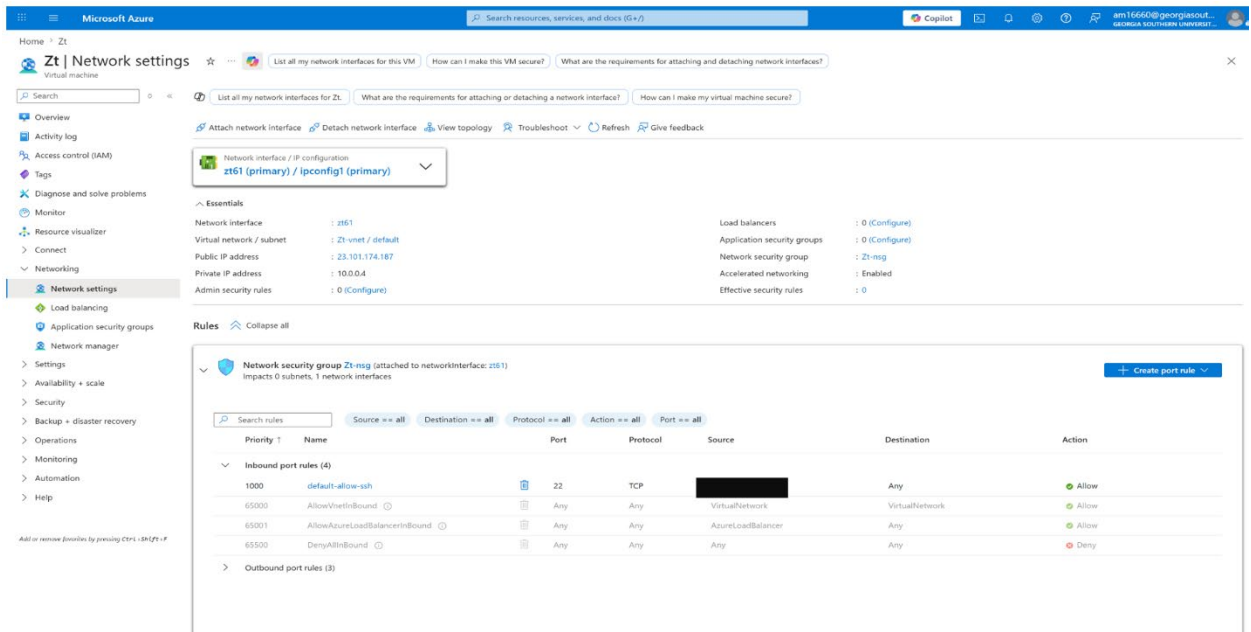


Figure 2. Restricted SSH inbound rule for Azure VM administration.

This screenshot shows the Network Security Group rule used to secure SSH administration. Port 22 remained available for remote access, but the source was restricted from Any to a trusted public IP address using /32 notation. This reduced unnecessary exposure while still allowing controlled administrative access to the Ubuntu VM.

Zero Trust Simulation Design

The Python simulation modeled users, devices, protected resources, network segments, access requests, a policy decision engine, a monitoring engine, and CSV output logs. The simulated users represented common enterprise roles, including HR, Finance, IT Administration, Guest, and an inactive former employee. The simulated devices represented managed, unmanaged, compliant, noncompliant, encrypted, and high-risk devices. The protected resources represented HR records, a Finance database, an administrative console, and a public portal.

The policy engine evaluated each access request using user identity, account status, role authorization, MFA requirement, device ownership, device management status, device compliance status, device encryption status, device risk score, source segment authorization, and resource sensitivity. The monitoring engine inspected each decision and generated alerts for denied requests. It also generated higher severity alerts when a device triggered repeated denied access attempts.

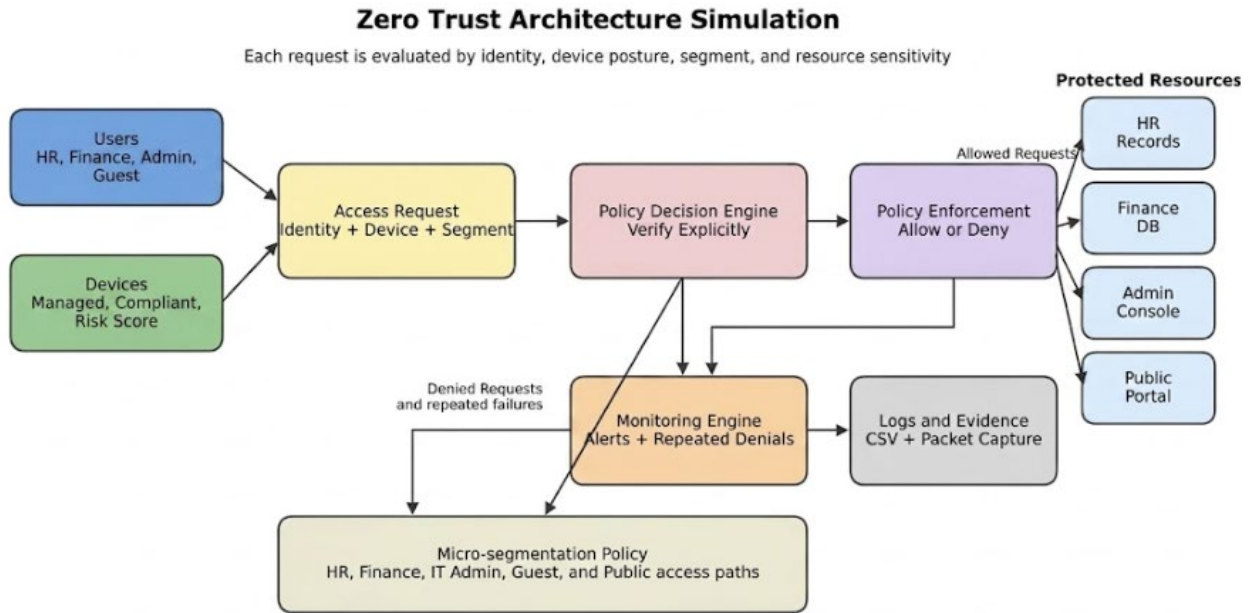


Figure 3. Zero Trust architecture design for the simulation.

This diagram shows how users and devices submit access requests to the policy decision engine. The engine evaluates identity, device posture, segment, and resource sensitivity before allowing or denying access to protected resources. Denied requests and repeated failures are sent to the monitoring engine and recorded as logs and alerts.

Implementation

The implementation was completed in two phases. The first phase established the Azure Ubuntu lab and generated real network evidence. The second phase implemented the Python-based Zero Trust access-control simulation.

Implementation Steps

1. Created an Azure Ubuntu 24.04 virtual machine.
2. Created supporting Azure resources, including the virtual network, subnet, network interface, public IP address, and Network Security Group.
3. Restricted SSH administration on port 22 to a trusted public IP address using /32 notation.
4. Connected to the VM through SSH using a private key.
5. Updated the Ubuntu environment and verified required tools.
6. Installed or verified nmap, tcpdump, Python 3, and Python package support.
7. Created a local protected HTTP resource on port 8080.
8. Generated controlled traffic using curl and nmap.
9. Captured traffic using tcpdump and saved the output as a .pcap file.

10. Implemented the Python Zero Trust simulation.
11. Ran the simulation and generated structured CSV outputs.
12. Reviewed the outputs to verify policy decisions, alerts, and test results.

Azure Ubuntu VM Implementation

An Azure Ubuntu 24.04 VM was created under the Azure for Students subscription. The VM was deployed with a virtual network, subnet, network interface, public IP address, and Network Security Group. After deployment, SSH access was configured using a private key. The SSH inbound rule was restricted to a specific trusted public IP address using /32 notation. This reduced the VM's exposure to the public internet while preserving secure administrative access.

After connecting to the VM, the environment was updated and security tools were installed or verified. The tools included nmap, tcpdump, Python 3, and Python's built-in HTTP server module. The system update check confirmed that the running kernel was up to date and that no services required restart. Tool verification output confirmed that nmap and tcpdump were available in the VM environment.

Protected Resource Implementation

A small protected test resource was created inside the VM. The project created a directory structure under `~/zt_lab`, then placed a simple `index.html` file inside a local web folder. Python's built-in HTTP server was started on port 8080. The resource was tested with curl against `127.0.0.1:8080`, which confirmed that the VM successfully served the protected resource locally.

This protected resource was not intended to represent a production web application. Instead, it served as a controlled resource for generating and capturing test traffic. This allowed the project to provide real packet-capture evidence without scanning or targeting any external system.

```
azureuser@Zt:~$ mkdir -p ~/zt_lab/evidence ~/zt_lab/www
azureuser@Zt:~$ echo "Zero Trust protected test resource - Ubuntu VM" > ~/zt_lab/www/index.html
azureuser@Zt:~$ cd ~/zt_lab/www
python3 -m http.server 8080 > ~/zt_lab/evidence/http_server.log 2>&1 &
[1] 3595
azureuser@Zt:~/zt_lab/www$ curl http://127.0.0.1:8080
Zero Trust protected test resource - Ubuntu VM
azureuser@Zt:~/zt_lab/www$
```

Figure 4. Protected local resource hosted on the Azure Ubuntu VM.

This screenshot shows the creation of the evidence folder, the protected test resource, and the Python HTTP server running on port 8080. The curl command confirms that the VM successfully served the protected resource from localhost.

Packet Capture Implementation

The project used tcpdump to capture traffic on the Azure Ubuntu VM. Controlled traffic was generated using curl and nmap against the protected resource on port 8080. The packet capture was saved as a .pcap file in the evidence directory. The saved capture file was then read back with tcpdump, showing TCP traffic to port 8080, an HTTP GET request, and a successful HTTP/1.0 200 OK response.

This step provided real network monitoring evidence. It demonstrated that the VM could generate, capture, and inspect traffic as part of the project's continuous monitoring evidence.

Python Simulation Implementation

The Python simulation was implemented in zero_trust_simulation.py. The code used structured classes to represent users, devices, resources, access requests, the policy engine, and the monitoring engine. The main simulation created predefined users, devices, and resources, then evaluated 13 test scenarios.

The policy engine returned a decision of ALLOW or DENY for each request. It also recorded the reason for the decision. If the actual decision matched the expected decision, the test was marked as PASS. The monitoring engine generated alerts for each denied request and raised high severity alerts for repeated denied access attempts.

The simulation generated three CSV output files: access_log.csv, alerts.csv, and test_summary.csv. The first records detailed access-decision history, the second records monitoring alerts, and the third records expected decisions, actual decisions, and pass/fail status.

Configuration and Command Examples

The Azure configuration used an inbound Network Security Group rule for SSH administration. The rule allowed TCP port 22 but restricted the source from Any to a trusted public IP address using /32 notation. This acted as a secure administration control and reduced unnecessary exposure of the VM.

```
mkdir -p ~/zt_lab/evidence ~/zt_lab/www
echo "Zero Trust protected test resource - Ubuntu VM" > ~/zt_lab/www/index.html
cd ~/zt_lab/www
python3 -m http.server 8080 > ~/zt_lab/evidence/http_server.log 2>&1 &
curl http://127.0.0.1:8080
```

```
sudo timeout 30 tcpdump -i any -nn 'tcp port 8080 or icmp' -w
~/zt_lab/evidence/zt_capture.pcap &
curl http://127.0.0.1:8080
nmap -sV -p 8080 127.0.0.1 | tee ~/zt_lab/evidence/nmap_8080.txt
```

```
sudo tcpdump -nn -r ~/zt_lab/evidence/zt_capture.pcap | head -20
```

```
python3 src/zero_trust_simulation.py
```

Secure Coding Techniques Applied

The simulation used structured classes for users, devices, resources, access requests, the policy engine, and the monitoring engine. This separation made the code easier to inspect and reduced the chance of mixing unrelated responsibilities. The policy engine evaluated multiple conditions before granting access instead of relying on one broad trust rule. Each decision included a reason, which made the output auditable. The monitoring engine generated alerts for denied access and repeated failures, which connected access control to continuous monitoring.

```
if user.role not in resource.allowed_roles:
    reasons.append("User role is not authorized for this resource")

if resource.requires_managed_device and not device.managed:
    reasons.append("Managed device is required")

if request.source_segment not in allowed_sources:
    reasons.append("Source segment is blocked by micro-segmentation policy")

decision = "ALLOW" if not reasons else "DENY"
```

This logic demonstrates least privilege, device posture checking, and micro-segmentation. The simulation only allowed access when all required conditions were satisfied.

Testing & Evaluation

Testing was performed in two parts. The first part tested network traffic generation and packet capture inside the Azure Ubuntu VM. The second part tested Zero Trust access-control logic through the Python simulation.

Test Case Categories

The testing plan included controlled network testing and policy simulation testing. The project did not perform offensive testing against external systems. All scans and packet captures were limited to the Azure Ubuntu VM and its local protected resource.

Test Category	Tool / Method	Purpose
Service detection test	nmap -sV -p 8080 127.0.0.1	Verified that the protected test resource was reachable on port 8080
Packet capture test	tcpdump	Captured and inspected traffic generated against the local resource
HTTP response test	curl	Confirmed that the protected local resource responded successfully
Access-control policy test	Python simulation	Evaluated whether users/devices should be allowed or denied
Monitoring and alert test	Python monitoring engine	Generated alerts for denied requests and repeated failures
Output validation test	CSV review	Verified access logs, alert logs, and test summary results

Detection and Incident Response Mechanisms

The project included two monitoring layers. At the network level, tcpdump captured traffic generated against the protected resource. This provided packet-level visibility into the test environment and confirmed that HTTP traffic could be inspected after capture. At the simulation level, the monitoring engine generated alerts when access requests were denied. It also generated higher severity alerts when repeated denied attempts were detected from the same device.

13. Receive an access request.
14. Evaluate the request using identity, role, device posture, segment, and resource rules.
15. Deny the request if one or more policy conditions fail.
16. Log the denial with a reason.
17. Generate a monitoring alert.
18. Escalate repeated denied attempts to a high-severity alert.

This workflow models how a Zero Trust environment can support detection and response by connecting access-control decisions to auditable monitoring output.

Traffic Capture Test

To validate the monitoring portion of the cloud lab, a simple protected HTTP resource was hosted locally on the Azure Ubuntu VM using Python's built-in web server on port 8080. Controlled test traffic was generated with curl and nmap, while tcpdump captured packets on the

VM. The resulting packet capture showed TCP traffic to port 8080, an HTTP GET request, and an HTTP/1.0 200 OK response. This confirmed that the lab environment could generate, capture, and inspect real network traffic as part of continuous monitoring evidence.

```
azureuser@Zt:~/zt_lab/www$ sudo timeout 30 tcpdump -i any -nn 'tcp port 8080 or icmp' -w ~/zt_lab/evidence/zt_capture.pcap &
[2] 3653
azureuser@Zt:~/zt_lab/www$ libibverbs: Warning: couldn't open config directory '/etc/libibverbs.d'.
tcpdump: data link type LINUX_SLL2
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
curl http://127.0.0.1:8080
Zero Trust protected test resource - Ubuntu VM
azureuser@Zt:~/zt_lab/www$ nmap -sV -p 8080 127.0.0.1 | tee ~/zt_lab/evidence/nmap_8080.txt12 packets captured
26 packets received by filter
0 packets dropped by kernel
ls -lh ~/zt_lab/evidence
total 8.0K
-rw-rw-r-- 1 azureuser azureuser 240 May  9 10:48 http_server.log
-rw-r--r-- 1 tcpdump tcpdump 1.4K May  9 10:48 zt_capture.pcap
[2]+  Exit 124                  sudo timeout 30 tcpdump -i any -nn 'tcp port 8080 or icmp' -w ~/zt_lab/evidence/zt_capture.pcap
```

Figure 5. Packet capture and controlled test traffic generation.

This screenshot shows tcpdump capturing traffic on the Azure Ubuntu VM while controlled HTTP traffic was generated against the protected resource on port 8080. The evidence directory confirms that the HTTP server log and packet capture file were created successfully.

```
azureuser@Zt:~/zt_lab/www$ sudo tcpdump -nn -r ~/zt_lab/evidence/zt_capture.pcap | head -20
reading from file /home/azureuser/zt_lab/evidence/zt_capture.pcap, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144
Warning: interface names might be incorrect
10:48:14.168227 lo In IP 127.0.0.1.34218 > 127.0.0.1.8080: Flags [S], seq 1320977220, win 65495, options [mss 65495,sackOK,TS val 2197598675 ecr 0,nop,wscale 7], length 0
10:48:14.168237 lo In IP 127.0.0.1.8080 > 127.0.0.1.34218: Flags [S.], seq 2833041329, ack 1320977221, win 65483, options [mss 65495,sackOK,TS val 2197598675 ecr 2197598675,nop,wscale 7], length 0
10:48:14.168248 lo In IP 127.0.0.1.34218 > 127.0.0.1.8080: Flags [.], ack 1, win 512, options [nop,nop,TS val 2197598675 ecr 2197598675], length 0
10:48:14.168307 lo In IP 127.0.0.1.34218 > 127.0.0.1.8080: Flags [P.], seq 1:78, ack 1, win 512, options [nop,nop,TS val 2197598675 ecr 2197598675], length 77: HTTP: GET / HTTP/1.1
10:48:14.168314 lo In IP 127.0.0.1.8080 > 127.0.0.1.34218: Flags [.], ack 78, win 511, options [nop,nop,TS val 2197598675 ecr 2197598675], length 0
10:48:14.168862 lo In IP 127.0.0.1.8080 > 127.0.0.1.34218: Flags [P.], seq 1:186, ack 78, win 512, options [nop,nop,TS val 2197598676 ecr 2197598675], length 185: HTTP: HTTP/1.0 200 OK
10:48:14.168880 lo In IP 127.0.0.1.34218 > 127.0.0.1.8080: Flags [.], ack 186, win 511, options [nop,nop,TS val 2197598676 ecr 2197598676], length 0
10:48:14.168912 lo In IP 127.0.0.1.8080 > 127.0.0.1.34218: Flags [P.], seq 186:233, ack 78, win 512, options [nop,nop,TS val 2197598676 ecr 2197598676], length 47: HTTP
10:48:14.168918 lo In IP 127.0.0.1.34218 > 127.0.0.1.8080: Flags [.], ack 233, win 511, options [nop,nop,TS val 2197598676 ecr 2197598676], length 0
10:48:14.168945 lo In IP 127.0.0.1.34218 > 127.0.0.1.8080: Flags [F.], seq 78, ack 233, win 512, options [nop,nop,TS val 2197598676 ecr 2197598676], length 0
10:48:14.168953 lo In IP 127.0.0.1.8080 > 127.0.0.1.34218: Flags [F.], seq 233, ack 79, win 512, options [nop,nop,TS val 2197598676 ecr 2197598676], length 0
10:48:14.168967 lo In IP 127.0.0.1.34218 > 127.0.0.1.8080: Flags [.], ack 234, win 512, options [nop,nop,TS val 2197598676 ecr 2197598676], length 0
```

Figure 6. Readable packet-capture evidence from tcpdump.

This screenshot shows readable packet-capture output from the saved .pcap file. The output includes TCP traffic to port 8080, an HTTP GET request, and a successful HTTP/1.0 200 OK response. This confirms that the VM generated and monitored real network traffic.

Zero Trust Policy Test Scenarios

The simulation evaluated 13 access requests. The scenarios included normal authorized access, unauthorized cross-segment access, noncompliant device access, high-risk unmanaged device access, repeated failed attempts, public portal access, and inactive account access.

Test ID	Scenario	Expected Result	Actual Result	Status
T01	HR user accesses HR records from compliant device	ALLOW	ALLOW	PASS
T02	HR user attempts Finance database access	DENY	DENY	PASS
T03	Finance user uses noncompliant device	DENY	DENY	PASS
T04	Admin accesses admin console from trusted segment	ALLOW	ALLOW	PASS
T05	Guest attempts internal HR access	DENY	DENY	PASS
T06	High-risk unmanaged device attempts admin access	DENY	DENY	PASS
T07	Valid HR user attempts access from wrong segment	DENY	DENY	PASS
T08A	Repeated failed attempt 1	DENY	DENY	PASS
T08B	Repeated failed attempt 2	DENY	DENY	PASS
T08C	Repeated failed attempt 3	DENY	DENY	PASS
T09	Device remains noncompliant after prior denial	DENY	DENY	PASS

T10	Guest accesses public portal	ALLOW	ALLOW	PASS
T11	Inactive account attempts access	DENY	DENY	PASS

Metric	Result
Total access requests	13
Allowed requests	3
Denied requests	10
Policy tests passed	13/13
Alerts generated	13

Result Type	Count
Allowed access requests	3
Denied access requests	10
Generated alerts	13

The results table summarizes the simulation output by comparing allowed requests, denied requests, and generated alerts.

```

Last login: Sat May  9 22:06:17 2026 from [REDACTED]
azureuser@Zt:~$ cd ~/zt_lab
azureuser@Zt:~/zt_lab$ ls -lh src
total 4.0K
-rw----- 1 azureuser azureuser 125 May  9 22:28 zero_trust_simulation.py.save
azureuser@Zt:~/zt_lab$ rm src/zero_trust_simulation.py.save
azureuser@Zt:~/zt_lab$ nano src/zero_trust_simulation.py
azureuser@Zt:~/zt_lab$ python3 src/zero_trust_simulation.py
Zero Trust simulation complete
Total access requests: 13
Allowed requests: 3
Denied requests: 10
Policy tests passed: 13/13
Alerts generated: 13
Results saved in: /home/azureuser/zt_lab/results
azureuser@Zt:~/zt_lab$

```

Figure 7. Zero Trust simulation execution summary.

This screenshot shows the Python-based Zero Trust simulation running successfully inside the Azure Ubuntu VM. The simulation evaluated 13 access requests using identity, role, device compliance, source segment, resource sensitivity, and monitoring rules. All 13 policy tests passed, with 3 requests allowed, 10 requests denied, and 13 alerts generated.

```

azureuser@Zt:~/zt_lab$ cat results/test_summary.csv
request_id,scenario,expected_decision,actual_decision,policy_result
T01,HR user accesses HR records from compliant device,ALLOW,ALLOW,PASS
T02,HR user attempts Finance database access,DENY,DENY,PASS
T03,Finance user uses noncompliant device,DENY,DENY,PASS
T04,Admin accesses admin console from trusted segment,ALLOW,ALLOW,PASS
T05,Guest attempts internal HR access,DENY,DENY,PASS
T06,High-risk unmanaged device attempts admin access,DENY,DENY,PASS
T07,Valid HR user attempts access from wrong segment,DENY,DENY,PASS
T08A,Repeated failed attempt 1,DENY,DENY,PASS
T08B,Repeated failed attempt 2,DENY,DENY,PASS
T08C,Repeated failed attempt 3,DENY,DENY,PASS
T09,Device remains noncompliant after prior denial,DENY,DENY,PASS
T10,Guest accesses public portal,ALLOW,ALLOW,PASS
T11,Inactive account attempts access,DENY,DENY,PASS

```

Figure 8. Zero Trust policy test summary.

This screenshot shows the test_summary.csv output. Each test case lists the access scenario, expected decision, actual decision, and policy result. The output confirms that the simulated Zero Trust policy engine correctly allowed authorized requests and denied unauthorized or risky requests.

```

azureuser@Zt:~/zt_lab$ cat results/alerts.csv
timestamp,request_id,severity,alert_type,message
2026-05-09T22:38:40.734972+00:00,T02,MEDIUM,POLICY_DENIAL,Access denied for u_hr to FINANCE_DB: User
role is not authorized for this resource; Source segment is blocked by micro-segmentation policy
2026-05-09T22:38:40.734981+00:00,T03,MEDIUM,POLICY_DENIAL,Access denied for u_fin to FINANCE_DB: Dev
ice is not compliant
2026-05-09T22:38:40.734993+00:00,T05,MEDIUM,POLICY_DENIAL,Access denied for u_guest to HR_RECORDS: U
ser role is not authorized for this resource; MFA is required but not enabled; Managed device is req
uired; Encrypted device is required for sensitive resource; Source segment is blocked by micro-segme
ntation policy
2026-05-09T22:38:40.734998+00:00,T06,MEDIUM,POLICY_DENIAL,Access denied for u_guest to ADMIN_CONSOLE
: User role is not authorized for this resource; MFA is required but not enabled; Managed device is
required; Device is not compliant; Encrypted device is required for sensitive resource; Device risk
score is too high; Source segment is blocked by micro-segmentation policy
2026-05-09T22:38:40.735003+00:00,T07,MEDIUM,POLICY_DENIAL,Access denied for u_hr to HR_RECORDS: Sour
ce segment is blocked by micro-segmentation policy
2026-05-09T22:38:40.735006+00:00,T08A,MEDIUM,POLICY_DENIAL,Access denied for u_guest to FINANCE_DB:
User role is not authorized for this resource; MFA is required but not enabled; Managed device is re
quired; Device is not compliant; Encrypted device is required for sensitive resource; Device risk sc
ore is too high; Source segment is blocked by micro-segmentation policy
2026-05-09T22:38:40.735011+00:00,T08B,MEDIUM,POLICY_DENIAL,Access denied for u_guest to FINANCE_DB:
User role is not authorized for this resource; MFA is required but not enabled; Managed device is re
quired; Device is not compliant; Encrypted device is required for sensitive resource; Device risk sc
ore is too high; Source segment is blocked by micro-segmentation policy
2026-05-09T22:38:40.735011+00:00,T08B,HIGH,REPEATED_DENIALS,Device d_unknown triggered repeated deni
ed access attempts
2026-05-09T22:38:40.735017+00:00,T08C,MEDIUM,POLICY_DENIAL,Access denied for u_guest to FINANCE_DB:
User role is not authorized for this resource; MFA is required but not enabled; Managed device is re
quired; Device is not compliant; Encrypted device is required for sensitive resource; Device risk sc
ore is too high; Source segment is blocked by micro-segmentation policy
2026-05-09T22:38:40.735017+00:00,T08C,HIGH,REPEATED_DENIALS,Device d_unknown triggered repeated deni
ed access attempts
2026-05-09T22:38:40.735022+00:00,T09,MEDIUM,POLICY_DENIAL,Access denied for u_fin to FINANCE_DB: Dev
ice is not compliant
2026-05-09T22:38:40.735028+00:00,T11,MEDIUM,POLICY_DENIAL,Access denied for u_disabled to HR_RECORDS
: Inactive user account; Device is not assigned to requesting user
2026-05-09T22:38:40.735028+00:00,T11,HIGH,REPEATED_DENIALS,Device d_hr_laptop triggered repeated deni
ed access attempts

```

Figure 9. Monitoring alerts generated by the Zero Trust simulation.

This screenshot shows the alerts.csv output created by the monitoring engine. The alerts document policy denials and repeated denied access attempts, demonstrating how the simulation supports continuous monitoring and flags suspicious access behavior.

Results & Discussion

The results show that the project met its main goal: building a controlled Zero Trust lab that could enforce access decisions, generate alerts, and produce evidence from real network traffic. The Network Security Group demonstrated secure remote administration by restricting SSH access to a trusted source IP instead of leaving port 22 open to all internet addresses. The protected resource and packet capture test demonstrated real traffic generation and monitoring.

Objective Alignment

Project Objective	Evidence of Completion
Design a segmented enterprise-style Zero Trust model	The simulation modeled HR, Finance, IT Admin, Guest, and Public access paths through source-segment policy checks
Implement strong authentication and access-control logic	The policy engine evaluated user role, account status, MFA requirement, device ownership, and resource authorization
Model device compliance and endpoint trust	The simulation checked whether devices were managed, compliant, encrypted, and below the risk threshold
Demonstrate continuous monitoring	The monitoring engine generated alerts for policy denials and repeated denied attempts
Produce real network evidence	The Azure Ubuntu VM generated and captured traffic with curl, nmap, and tcpdump
Document implementation and results	The project produced screenshots, CSV output files, source code, and a live demonstration recording

The Python simulation evaluated 13 access requests across multiple Zero Trust scenarios. The test cases included authorized access, unauthorized cross-segment access, noncompliant devices, high-risk unmanaged devices, repeated failed attempts, guest access, and inactive account access. The simulation produced three output files: `access_log.csv`, `alerts.csv`, and `test_summary.csv`. All 13 policy tests passed. The final results included 3 allowed requests, 10 denied requests, and 13 generated alerts. These results show that the simulated policy engine enforced least privilege, device compliance, micro-segmentation, and continuous monitoring rules.

The allowed requests were cases where the user, device, segment, and resource relationship satisfied the policy. For example, the HR user could access HR records from a compliant device, the administrator could access the administrative console from the trusted IT Admin segment, and the guest user could access the public portal. These allowed cases show that the system did not deny access blindly. It allowed access when the request met the policy requirements.

The denied requests were equally important. The simulation denied HR access to Finance resources, guest access to internal HR records, noncompliant device access, high-risk unmanaged device access, wrong-segment access, and inactive account access. These denials demonstrate

least privilege, device compliance enforcement, micro-segmentation, and account-status validation.

The monitoring output also supported the Zero Trust model. The simulation generated 13 alerts, including policy-denial alerts and repeated-denial alerts. These alerts show that the system did not only make access decisions. It also created an audit trail for security review. In a real enterprise, similar alerts could be sent to a Security Information and Event Management system for investigation and correlation with other events.

Limitations, Challenges, and Potential Improvements

The project also has limitations. The Python simulation does not replace a production Zero Trust platform. It does not integrate with a real identity provider, endpoint detection platform, multifactor authentication service, or enterprise SIEM. The packet capture was performed against a local protected resource rather than across multiple production network segments. However, these limitations are appropriate for a controlled course project. The implementation still demonstrates the core logic of Zero Trust access enforcement and monitoring.

A major challenge was the limited availability of preconfigured security lab environments. Kali Linux, Ubuntu course servers, and GNS3 resources were not available through the original class environment, so the project was adapted to a Microsoft Azure Ubuntu VM. This change preserved the core project goals while allowing the final implementation to include real Linux access, cloud configuration evidence, packet capture, and reproducible simulation results.

Potential improvements include expanding the lab into multiple Azure VMs, separating HR, Finance, IT Admin, and Guest resources into different subnets, integrating a real identity provider, forwarding alerts into a SIEM dashboard, and adding an IDS tool such as Snort or Suricata. The simulation could also be improved by adding dynamic risk scoring, session expiration, adaptive authentication, and policy updates based on monitoring events.

Conclusion & Future Work

This project designed and simulated a Zero Trust Architecture for secure enterprise networking. The work combined an Azure Ubuntu cloud lab, secure SSH configuration, Network Security Group rules, packet capture, and a Python-based Zero Trust policy simulation. The project demonstrated micro-segmentation, least privilege, device compliance checks, role-based authorization, account-status validation, continuous monitoring, and alert generation.

Key Findings

The project produced three key findings. First, restricting administrative access through the Azure Network Security Group showed how a basic cloud security control can reduce unnecessary exposure while still allowing secure remote management. Second, the packet-

capture test showed that the lab environment could generate, capture, and inspect real network traffic. Third, the Python simulation showed that Zero Trust access-control logic can be modeled in a clear and auditable way by checking identity, role, device posture, source segment, resource sensitivity, and monitoring behavior before allowing access.

The final simulation evaluated 13 access requests. All 13 policy tests passed. The simulation allowed 3 requests, denied 10 requests, and generated 13 alerts. These results show that the policy engine correctly enforced the intended Zero Trust rules and produced auditable monitoring evidence.

Future work could expand the project in several ways. First, the simulation could be connected to a real identity provider such as Microsoft Entra ID, Okta, or another authentication platform. Second, device compliance could be integrated with endpoint management tools. Third, monitoring alerts could be forwarded to a SIEM platform for dashboarding and correlation. Fourth, the lab could be expanded into a multi-VM environment with separate HR, Finance, IT Admin, and Guest segments. Finally, additional tools such as Snort, Suricata, or GNS3 could be added to support intrusion detection and more advanced network simulation.

Scaling the Project

At a larger scale, this project could evolve from a single-VM academic prototype into a multi-segment enterprise lab. Separate VMs or containers could represent HR systems, finance systems, administrative servers, user endpoints, and guest devices. Azure subnets and Network Security Groups could enforce network-level segmentation, while the Python policy engine could be expanded into an API-based policy decision service. Alerts could be forwarded into a SIEM, and packet captures could be replaced or supplemented with IDS alerts from Snort or Suricata. This would make the project closer to a real Zero Trust deployment while preserving the same core design principle: no user, device, or segment receives automatic trust.

Overall, this project shows how Zero Trust principles can be turned into practical access-control logic and tested through both cloud lab evidence and reproducible simulation output.

References

- Cisco Systems. (2018). The zero trust extended ecosystem: Building a comprehensive approach to cybersecurity. Cisco.
https://www.cisco.com/c/dam/m/en_sg/solutions/security/pdfs/forrester-ztx.pdf
- Cisco Systems. (2024). Cisco zero trust architecture guide. Cisco.
<https://www.cisco.com/c/en/us/solutions/collateral/enterprise/design-zone-security/zt-ag.html>

- Fernandez, E. B., & Brazhuk, A. (2022). A critical analysis of zero trust architecture (ZTA). SSRN Electronic Journal.
https://www.researchgate.net/publication/363306732_A_Critical_Analysis_of_Zero_Trust_Architecture_Zta
- Google. (2014). BeyondCorp: A new approach to enterprise security. Google Research.
<https://research.google/pubs/beyondcorp-a-new-approach-to-enterprise-security/>
- Microsoft Corporation. (2023). Zero trust adoption framework. Microsoft Learn.
<https://learn.microsoft.com/en-us/security/zero-trust/adopt/zero-trust-adoption-overview>
- National Institute of Standards and Technology. (2020). Zero trust architecture (NIST Special Publication 800-207). U.S. Department of Commerce.
<https://nvlpubs.nist.gov/nistpubs/specialpublications/NIST.SP.800-207.pdf>
- Microsoft Corporation. (2026). Quickstart: Create a Linux virtual machine in the Azure portal. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/virtual-machines/linux/quick-create-portal>
- Microsoft Corporation. (2025). Connect to a Linux VM. Microsoft Learn.
<https://learn.microsoft.com/en-us/azure/virtual-machines/linux-vm-connect>
- Microsoft Corporation. (2024). Use SSH keys to connect to Linux VMs. Microsoft Learn.
<https://learn.microsoft.com/en-us/azure/virtual-machines/linux/ssh-from-windows>
- Nmap Project. (n.d.). Nmap: The Network Mapper. <https://nmap.org/>
- OWASP Foundation. (n.d.). Secure product design cheat sheet. OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/Secure_Product_Design_Cheat_Sheet.html
- Python Software Foundation. (2026). http.server, HTTP servers. Python documentation.
<https://docs.python.org/3/library/http.server.html>
- tcpdump Group. (n.d.). tcpdump manual page. <https://www.tcpdump.org/>

Appendix

Appendix A. Screenshot Evidence Packet

Appendix A. Screenshot Evidence Packet

Attach the screenshot evidence document containing the Azure deployment, resource list, restricted SSH rule, successful SSH connection, system update and tool verification, protected

resource test, packet-capture evidence, VM overview page, simulation execution summary, generated result files, policy test summary, monitoring alerts, and access log output.

Screenshot #	Evidence Shown
1	Azure Ubuntu VM deployment completed successfully
2	Azure resources created for the lab, including VM, network interface, Network Security Group, virtual network, and public IP
3	SSH inbound rule restricted from Any to a trusted public IP address using /32 notation
4	Successful SSH connection to the Azure Ubuntu VM
5	System update check and tool verification for nmap and tcpdump
6	Protected local resource created and served through Python HTTP server on port 8080
7	Packet capture started with tcpdump, test traffic generated with curl and nmap, and .pcap file created
8	Readable packet-capture output showing HTTP GET request and HTTP/1.0 200 OK response
9	Zero Trust simulation execution summary showing 13 requests, 3 allowed, 10 denied, 13/13 tests passed, and 13 alerts
10	Generated result files: access_log.csv, alerts.csv, and test_summary.csv
11	Zero Trust policy test summary from test_summary.csv
12	Monitoring alerts generated by the simulation from alerts.csv
13	Monitoring alerts generated by the simulation from alerts.csv
14	Beginning of the Zero Trust access decision log from access_log.csv

Appendix A. Screenshot Evidence Packet

A separate screenshot evidence packet is attached with the final submission. The packet includes numbered screenshots showing the Azure VM deployment, Azure resource creation, restricted SSH rule, successful SSH connection, system update and tool verification, protected resource test, packet-capture evidence, simulation execution summary, generated result files, policy test summary, monitoring alerts, and access decision log.

Appendix B. Source Code Package

The source code package is submitted as a separate zipped folder with the final project files. The package includes the Python Zero Trust simulation, generated CSV output files, packet-capture evidence, and supporting lab files.

Appendix C. Live Demonstration Recording

The live demonstration recording is submitted as a separate video file. The recording shows the Zero Trust simulation running inside the Azure Ubuntu VM, generating the result files, and displaying the access-control and monitoring outputs.

Appendix E. Code Documentation

The main source file is `src/zero_trust_simulation.py`. The file contains data classes for User, Device, Resource, and AccessRequest; ZeroTrustPolicyEngine for access evaluation; MonitoringEngine for alert generation; test data for users, devices, resources, and scenarios; and CSV export logic for access logs, alerts, and test summaries. The source code package should include the Python script, generated CSV files, packet-capture files, and supporting lab evidence.